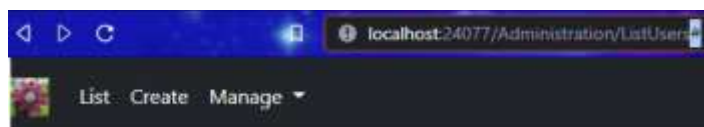


85 - Edit identity user

Към момента не сме имплементирали бутони Edit, Delete. Когато натиснем Edit, в адрес бара просто се добавя #



All Users

Add new user

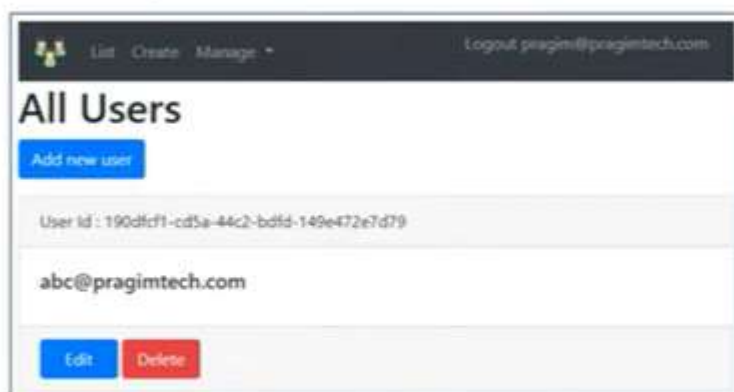
User Id : 1030daee-596d-414e-87d3-cba6c7e5935e

julia@abv.bg

Edit

Delete

Edit Identity User in ASP.NET Core



```
<a asp-action="EditUser" asp-controller="Administration"
  asp-route-id="@user.Id" class="btn btn-primary">
  Edit
</a>
```

Просто сменяме кода:

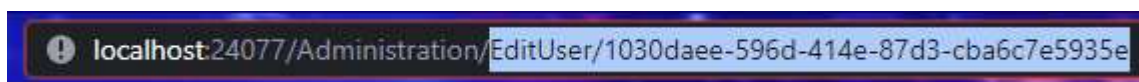
```
AccountController.cs  ListUsers.cshtml  _Layout.cshtml
<h5 class="card-title">@user.UserName</h5>
</div>
<div class="card-footer">
  <a href="#" class="btn btn-danger">Edit</a>
  <a href="#" class="btn btn-danger">Delete</a>
</div>
```

За да пренасочим потребителя към адрес, подавайки id

```
AccountController.cs | ListUsers.cshtml* | _Layout.cshtml | AdministrationControl
<h5 class="card-title">@user.UserName</h5>
</div>
<div class="card-footer">
  <a asp-action="EditUser" asp-controller="Administration"
    asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
  <a href="#" class="btn btn-danger">Delete</a>
</div>
```

```
<a asp-action="EditUser" asp-controller="Administration"
  asp-route-id="@user.Id" class="btn btn-primary">Edit</a>
```

Сега, при Edit, адресът автоматично прихваща id, но връща 404, защото все още не сме имплементирали EditUser в AdministrationController:



```
AdministrationController.cs* | AccountController.cs | ListUsers.cshtml | _Layout.cshtml
EmployeeManagement.Controllers.AdministrationController
[HttpGet]
0 references
public async Task<IActionResult> EditUser(string id)
{
    var user = await userManager.FindByIdAsync(id);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
        return View("NotFound");
    }

    // GetClaimsAsync returns the list of user Claims
    var userClaims = await userManager.GetClaimsAsync(user);
    // GetRolesAsync returns the list of user Roles
    var userRoles = await userManager.GetRolesAsync(user);

    var model = new EditUserViewModel
    {
        Id = user.Id,
        Email = user.Email,
        UserName = user.UserName,
        City = user.City,
        Claims = userClaims.Select(c => c.Value).ToList(),
        Roles = userRoles
    };
}
```

```
[HttpGet]
public async Task<IActionResult> EditUser(string id)
{
    var user = await userManager.FindByIdAsync(id);
```

```

if (user == null)
{
    ViewBag.ErrorMessage = $"User with Id = {id} cannot be found";
    return View("NotFound");
}

// GetClaimsAsync returns the list of user Claims
var userClaims = await userManager.GetClaimsAsync(user);
// GetRolesAsync returns the list of user Roles
var userRoles = await userManager.GetRolesAsync(user);

var model = new EditUserViewModel
{
    Id = user.Id,
    Email = user.Email,
    UserName = user.UserName,
    City = user.City,
    Claims = userClaims.Select(c => c.Value).ToList(),
    Roles = userRoles
};

return View(model);
}

```

Edit Identity User in ASP.NET Core

The screenshot shows a web application interface for editing a user. At the top, there's a navigation bar with 'Edit', 'Create', and 'Manage' links, and a user profile 'logout: pragim@pragimtech.com'. The main heading is 'Edit User'. Below this, there are four input fields: 'Id' (with a GUID value), 'Email' (pragim@pragimtech.com), 'Username' (pragim@pragimtech.com), and 'City' (empty). There are 'Update' and 'Cancel' buttons below the inputs. Below the inputs, there are two sections: 'User Roles' and 'User Claims'. Both sections show 'None at the moment' and a 'Manage' button.

Properties:

EditUserViewModel

```
public class EditUserViewModel
{
    public EditUserViewModel()
    {
        initialize    Claims = new List<string>(); Roles = new List<string>();
    }

    properties:      public string Id { get; set; }

                    [Required]
                    public string UserName { get; set; }

    validate         [Required][EmailAddress]
                    public string Email { get; set; }

                    public string City { get; set; }

                    public List<string> Claims { get; set; }

    interface        public IList<string> Roles { get; set; }
}
```

```
public class EditUserViewModel
{
    public EditUserViewModel()
    {
        Claims = new List<string>();
        Roles = new List<string>();
    }

    public string Id { get; set; }

    [Required]
    public string UserName { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    public string City { get; set; }

    public List<string> Claims { get; set; }

    public IList<string> Roles { get; set; }
}
```

Administration + EditUser.cshtml

```
@model EditUserViewModel
```

```
@{
    ViewBag.Title = "Edit User";
}
```

```
<h1>Edit User</h1>
```

```

<form method="post" class="mt-3">
  <div class="form-group row">
    <label asp-for="Id" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="Id" disabled class="form-control">
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="Email" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="Email" class="form-control">
      <span asp-validation-for="Email" class="text-danger"></span>
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="UserName" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="UserName" class="form-control">
      <span asp-validation-for="UserName" class="text-danger"></span>
    </div>
  </div>
  <div class="form-group row">
    <label asp-for="City" class="col-sm-2 col-form-label"></label>
    <div class="col-sm-10">
      <input asp-for="City" class="form-control">
    </div>
  </div>

  <div asp-validation-summary="All" class="text-danger"></div>

  <div class="form-group row">
    <div class="col-sm-10">
      <button type="submit" class="btn btn-primary">Update</button>
      <a asp-action="ListUsers" class="btn btn-primary">Cancel</a>
    </div>
  </div>

  <div class="card">
    <div class="card-header">
      <h3>User Roles</h3>
    </div>
    <div class="card-body">
      @if (Model.Roles.Any())
      {
        foreach (var role in Model.Roles)
        {
          <h5 class="card-title">@role</h5>
        }
      }
      else
      {
        <h5 class="card-title">None at the moment</h5>
      }
    </div>
    <div class="card-footer">

```

```

        <a href="#" style="width:auto" class="btn btn-primary">
            Manage Roles
        </a>
    </div>
</div>

<div class="card mt-3">
    <div class="card-header">
        <h3>User Claims</h3>
    </div>
    <div class="card-body">
        @if (Model.Claims.Any())
        {
            foreach (var claim in Model.Claims)
            {
                <h5 class="card-title">@claim</h5>
            }
        }
        else
        {
            <h5 class="card-title">None at the moment</h5>
        }
    </div>
    <div class="card-footer">
        <a href="#" style="width:auto" class="btn btn-primary">
            Manage Claims
        </a>
    </div>
</div>
</form>

```

В случая важното е, че <input> за Id не бива да се коригира, затова е оставено с disabled атрибут. Също не сме указали името на контролера, защото EditUser и ListUsers се управляват от него и се намират в папка Administration, така че това не е необходимо, а по желание. Имаме Model-object.Roles-property.Any-method.

```

[HttpPost]
public async Task<IActionResult> EditUser(EditUserViewModel model)
{
    var user = await userManager.FindByIdAsync(model.Id);

    if (user == null)
    {
        ViewBag.ErrorMessage = $"User with Id = {model.Id} cannot be found";
        return View("NotFound");
    }
    else
    {
        user.Email = model.Email;
        user.UserName = model.UserName;
        user.City = model.City;

        var result = await userManager.UpdateAsync(user);
    }
}

```

```

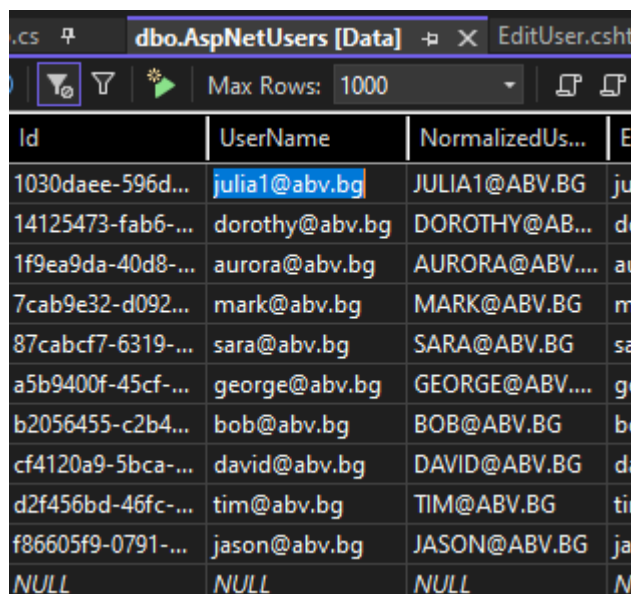
    if (result.Succeeded)
    {
        return RedirectToAction("ListUsers");
    }

    foreach (var error in result.Errors)
    {
        ModelState.AddModelError("", error.Description);
    }

    return View(model);
}
}

```

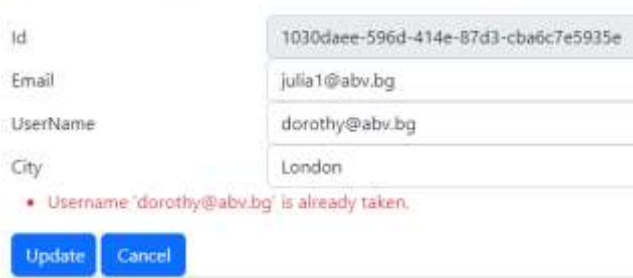
Проверяваме, че update работи:



Id	UserName	NormalizedUs...	Email
1030daee-596d-414e-87d3-cba6c7e5935e	julia1@abv.bg	JULIA1@ABV.BG	julia1@abv.bg
14125473-fab6-414e-87d3-cba6c7e5935e	dorothy@abv.bg	DOROTHY@ABV.BG	dorothy@abv.bg
1f9ea9da-40d8-414e-87d3-cba6c7e5935e	aurora@abv.bg	AURORA@ABV.BG	aurora@abv.bg
7cab9e32-d092-414e-87d3-cba6c7e5935e	mark@abv.bg	MARK@ABV.BG	mark@abv.bg
87cabcf7-6319-414e-87d3-cba6c7e5935e	sara@abv.bg	SARA@ABV.BG	sara@abv.bg
a5b9400f-45cf-414e-87d3-cba6c7e5935e	george@abv.bg	GEORGE@ABV.BG	george@abv.bg
b2056455-c2b4-414e-87d3-cba6c7e5935e	bob@abv.bg	BOB@ABV.BG	bob@abv.bg
cf4120a9-5bca-414e-87d3-cba6c7e5935e	david@abv.bg	DAVID@ABV.BG	david@abv.bg
d2f456bd-46fc-414e-87d3-cba6c7e5935e	tim@abv.bg	TIM@ABV.BG	tim@abv.bg
f86605f9-0791-414e-87d3-cba6c7e5935e	jason@abv.bg	JASON@ABV.BG	jason@abv.bg
NULL	NULL	NULL	NULL

The main difference between List and IList in C# is that **List is a class** that represents a list of objects which can be accessed by index while **IList is an interface** that represents a collection of objects which can be accessed by index.

Edit User



Id: 1030daee-596d-414e-87d3-cba6c7e5935e

Email: julia1@abv.bg

UserName: dorothy@abv.bg

City: London

• Username 'dorothy@abv.bg' is already taken.

Update Cancel

Имаме валидация при заето име по време на редакция.